

REST 練習 11 (延伸題) : Header/Item 單一 BAPI 呼叫 (銷售訂單模型)

這題刻意跳出 **SFLIGHT/SBOOK** 主題，改用銷售訂單 (**VBAP/VBAP**) 模型——原因見下方 Lecture 第二段，不是忘記維持課程主題一致性。

Lecture

rs10 教過「陣列輸入、逐筆呼叫 BAPI」——JSON Array 反序列化 Internal Table 後 **LOOP** 呼叫一支只吃單筆的 BAPI (**BAPI_FLBOOKING_CREATEFROMDATA**) N 次，10 筆輸入就呼叫 10 次 BAPI。但實務上還有另一種常見情境：一張單據本身就是「一個抬頭 (**Header**) 配多筆明細 (**Item**)」的結構，例如一張銷售訂單有一個客戶、一個訂單類型，但底下可能有 5 個不同的品項各訂不同數量——這種「Header 1 筆 + Item 多筆」的資料結構呼叫的 BAPI，通常介面本身就同時吃一個 Header 結構跟一個 Item 表格，一次呼叫就處理完整張單據，不是 **LOOP** 呼叫多次。這是跟 rs10 本質不同的整合模式：rs10 是「呼叫端自己迴圈呼叫單筆 BAPI N 次」，這題是「BAPI 介面本身支援 1 對多、呼叫端只呼叫一次」。

為什麼改用銷售訂單模型：這門課從 rs01 開始都沿用 **SFLIGHT/SBOOK/SCUSTOM** (航班訂位) 資料模型，但實際查證後發現，這個訓練資料模型裡沒有任何標準 BAPI 是「一次呼叫、Header(1)+Item(多筆)」的設計——**BAPI_FLIGHT_*** / **BAPI_FLBOOKING_*** 全部查過，訂位業務語意本來就是逐筆建立 (一個客戶訂一個航班的一個位子)，找不到「一次建立一個航班+多筆訂位」這種介面。銷售訂單 (**BAPI_SALESORDER_CREATEFROMDAT2**) 則是業界公認示範「Header+Item 一次 BAPI 呼叫」的標準教材，SAP 幾乎每一本 BAPI 整合教材都會用它當範例，值得專門為它跳出原本的主題。

這支 BAPI 出了名的「新手常踩坑」介面：它的 **IMPORTING/TABLES** 參數不是直接對應資料庫欄位，而是搭配一套「X 旗標結構」 (**ORDER_HEADER_INX/ORDER_ITEMS_INX**)——每個資料結構旁邊都有一個「哪些欄位要更新」的旗標結構，欄位設了值但沒有在對應的 X 旗標打勾，BAPI 可能完全忽略你填的值。這是 SAP 一系列「Header+Item」BAPI (不只銷售訂單，採購單 **BAPI_PO_CREATE1**、生產訂單等也都有類似設計) 的共通模式，一旦學會這個模式，日後遇到任何一支 BAPI 帶著 **XXX_IN/XXX_INX** 成對參數，就知道該怎麼下手。

這題示範一個額外的教學設計：把 BAPI 內建的 **TESTRUN** 旗標直接暴露成 REST API 的查詢參數 (? **testrun=true**)。TESTRUN='X' 時 BAPI 會照樣執行所有業務規則檢查、回傳 **RETURN** 訊息，但不會真的寫入資料庫——這正是官方文件建議「新手第一次串接這類 X 旗標複雜的 BAPI，先用小規模 testrun 驗證欄位對應對不對」的做法，這題直接把這個除錯手法做成 API 使用者也能用的功能：呼叫端可以先用 **testrun=true** 預覽「如果真的送出，BAPI 會不會接受這筆資料」，確認沒問題後再用正式呼叫 (不帶這個參數) 真的建立訂單。

學習目標

- 認識「Header(1)+Item(多筆)」單據型 BAPI 的介面設計模式：**XXX_HEADER_IN/XXX_HEADER_INX** (抬頭資料 + X 旗標) 配對、**XXX_ITEMS_IN/XXX_ITEMS_INX** (明細資料 + X 旗標) 配對的表格，一次呼叫處理整張單據
- 學會 **/UI2/CL_JSON** 反序列化兩層巢狀 JSON (**header** 物件底下同時有純量欄位跟一個子陣列 **items**，不是 **header/items** 並列同一層) 進 ABAP 巢狀結構，這是這門課第一次出現巢狀 JSON (之前都是扁平物件或扁平陣列)

- 理解 X 旗標結構的用途與新建立時的填法 (`UPDATEFLAG='I'` + 有填值的欄位對應打 'X')，並認識這類介面「新手常踩坑」的原因
- 認識把 BAPI 內建的 `TESTRUN` 機制暴露成 API 查詢參數的設計手法，作為「呼叫端可以先預覽、確認無誤再正式送出」的實務模式
- 分辨這題跟 rs10 兩種不同的「一次處理多筆」整合模式：**rs10 是呼叫端迴圈呼叫單筆 BAPI N 次；這題是 BAPI 介面本身一次吃 1 對多**

事前準備

ADT 端物件已由課程準備好 (`$TMP`)：

- `ZCX_RS11_SALESORDER_ERROR`——業務例外，同 rs07~rs10 設計
- `ZCL_RS11_SALESORDER_SERVICE`——公開
`ty_header/ty_item/tt_item/ty_create_request/ty_result` ; `validate_request` (純邏輯，附 ABAP Unit 測試，含一個驗證 `CL_JSON` 巢狀往返正確性的測試)、`create_sales_order` (呼叫 BAPI，不寫測試)
- `ZCL_RS11_APP`——Application Class，router 掛 `/salesorders` → `ZCL_RS11_SALESORDERS`；覆寫 `HANDLE_CSRF_TOKEN` 跳過 CSRF 檢查
- `ZCL_RS11_SALESORDERS`——薄層 Resource，只有 `POST` (這題聚焦「巢狀 JSON → 一次 BAPI 呼叫」，刻意不做完整 CRUD，沒有對應的 `GET` 單筆資源)

測試用主資料 (已用 ADT Data Preview freestyle SQL 查證真實存在)：

用途	值	說明
訂單類型 <code>docType</code>	TA	這個系統實測存在的標準訂單類型
銷售組織 <code>salesOrg</code>	1710	
配銷通路 <code>distrChan</code>	10	
產品組 <code>division</code>	00	
客戶 (Sold-to) <code>soldTo</code>	0017100002	客戶 <code>Domestic US Customer 2</code> ，銷售區域資料的價格群組 (<code>KNVV-KONDA</code>) 是空白，避開了 <code>USCU_L09</code> 那類「有設定 <code>KONDA=C1</code> 但系統未定義該價格群組」的客戶
物料 <code>material</code>	F-10A	成品 (<code>MTART=FERT</code>)，已確認在 <code>SALES ORG 1710/10</code> 底下有銷售資料、物料群組 <code>L004</code> 有在 <code>T023</code> 定義、 <code>MAKT</code> 有英文 (<code>SPRAS=E</code>) 物料說明——這三個條件都符合才不會被 BAPI 拒絕 (依序避開了 <code>MZ-FG-C900</code> 的「物料群組未定義」跟 <code>FG200</code> 的「只有非英文物料說明，缺英文說明」兩種資料缺口)
單位 <code>unit</code>	ST	件

題目需求 (對照已建好的答案物件)

1. 型別設計——**items** 巢狀在 **header** 底下 (不是跟 **header** 並列同一層) · 呼應「訂單抬頭底下才有品項明細」的業務語意：

```
``abap TYPES: BEGIN OF ty_item, material TYPE bapisditm-material, quantity TYPE bapisditm-target_qty, unit
TYPE bapisditm-target_qu, END OF ty_item, tt_item TYPE STANDARD TABLE OF ty_item WITH EMPTY KEY,

BEGIN OF ty_header, doc_type TYPE auart, sales_org TYPE vkorg, distr_chan TYPE vtweg, division TYPE spart,
sold_to TYPE kunnr, items TYPE tt_item, END OF ty_header,

BEGIN OF ty_create_request, header TYPE ty_header, END OF ty_create_request. ``
```

請求 JSON body 對應長這樣 (**items** 是 **header** 物件底下的一個屬性 · 兩層巢狀；示範帶兩筆 items)：

```
``json { "header": { "docType": "TA", "salesOrg": "1710", "distrChan": "10", "division":
"00", "soldTo": "0017100002", "items": [ { "material": "F-10A", "quantity": 10, "unit":
"ST" }, { "material": "F-10A", "quantity": 5, "unit": "ST" } ] } } ``

/UI2/CL_JSON=>DESERIALIZE 對「結構裡有純量欄位、又有一個 TABLE 型別子欄位」是直接支援的，
CHANGING data 餵一個這樣的巢狀變數即可，不需要分兩階段手動組裝——這點已經用 ABAP Unit 的往返測試
( ltc_json_roundtrip · 涵蓋兩筆 items ) 驗證過，不用等到掛 SICF 才確認。
```

1. **create_sales_order**——組 X 旗標結構 + 呼叫 BAPI (核心邏輯)：

```
``abap ls_header_in-doc_type = is_request-header-doc_type. ls_header_in-sales_org = is_request-header-
sales_org. ls_header_in-distr_chan = is_request-header-distr_chan. ls_header_in-division = is_request-header-
division.

ls_header_inx-updateflag = 'I'. ls_header_inx-doc_type = 'X'. ls_header_inx-sales_org = 'X'. ls_header_inx-
distr_chan = 'X'. ls_header_inx-division = 'X'.

APPEND VALUE #( partn_role = 'AG' partn_num = is_request-header-sold_to ) TO lt_partners.

LOOP AT is_request-header-items INTO DATA(ls_item). DATA(lv_itm_number) = CONV bapisditm-itm_number(
sy-tabix * 10 ).

APPEND VALUE #( itm_number = lv_itm_number material = ls_item-material target_qty = ls_item-quantity
target_qu = ls_item-unit ) TO lt_items_in. APPEND VALUE #( itm_number = lv_itm_number updateflag = 'I'
material = 'X' target_qty = 'X' target_qu = 'X' ) TO lt_items_inx. ENDLOOP.

CALL FUNCTION 'BAPI_SALESORDER_CREATEFROMDAT2' EXPORTING order_header_in = ls_header_in
order_header_inx = ls_header_inx testrun = COND bapiflag-bapiflag( WHEN iv_testrun = abap_true THEN 'X'
ELSE space ) IMPORTING salesdocument = lv_salesdocument TABLES return = lt_return order_items_in =
lt_items_in order_items_inx = lt_items_inx order_partners = lt_partners. ``
```

幾個實作細節：

- **ORDER_PARTNERS** 是必填的 **TABLES** 參數 (沒有 **OPTIONAL**) ——至少要給一筆 **PARTN_ROLE='AG'** (Sold-to Party) + **PARTN_NUMB**=客戶編號，不然 BAPI 大概率報錯；**ITM_NUMBER** 留白代表這個夥伴指派是抬頭層級 (不是某個特定項次專屬的夥伴)
- **項次編號 (**ITM_NUMBER**) 用 **sy-tabix * 10** 自動編號** (10, 20, 30...) · 比照 SAP 標準畫面的項次編號慣例 (項次之間留間隔，方便日後插入新項次)；**CONV bapisditm-**

`itm_number(...)` 是整數轉 NUMC，屬於安全的數值轉型，跟 rs07 教過的「字串轉 DATS 不會拋例外」是不同情境（那個是字串格式驗證問題，這裡是單純數字轉型）

- **Header 的 X 旗標** 本題選擇「每個有填值的欄位都明確打 'X'」，而不是只設 `updateflag='I'` 就好——這是比較保守、跨版本相容性更好的寫法，也是這支 BAPI 常見教學範例的做法；如果之後實測發現某個環境只設 `updateflag` 也能建立成功，不代表這裡的寫法錯，只是這支 BAPI 的容錯行為比想像中寬鬆
- **TESTRUN 分支** 收尾都要呼叫 **BAPI_TRANSACTION_ROLLBACK**（失敗時）或依 `iv_testrun` 決定 **COMMIT/ROLLBACK**（成功時）——即使 `testrun` 模式沒有真的寫入資料，官方文件仍建議呼叫 **ROLLBACK** 清除當次呼叫在系統內部累積的緩衝，這是收尾禮貌，不是必要動作

1. **ZCL_RS11_SALESORDERS~POST**——把 `testrun` 查詢參數接進來，並依照是否為 `testrun` 決定回應狀態碼：

```
``abap DATA(lv_testrun) = COND abap_bool( WHEN mo_request->has_uri_query_parameter( 'testrun' ) =
abap_true AND mo_request->get_uri_query_parameter( 'testrun' ) = 'true' THEN abap_true ELSE abap_false ).
```

```
DATA(ls_result) = zcl_rs11_salesorder_service=>create_sales_order( is_request = ls_request iv_testrun =
lv_testrun ).
```

```
mo_response->set_status( COND #( WHEN lv_testrun = abap_true THEN cl_rest_status_code=>gc_success_ok
ELSE cl_rest_status_code=>gc_success_created ) ). ``
```

`testrun=true` 回 **200 OK**（沒有建立新資源，只是預覽），正式建立回 **201 Created**（真的多了一筆銷售訂單）——這個狀態碼差異本身就是「這次呼叫有沒有真的改變系統狀態」的語意訊號。

SICF 掛載步驟

沿用既有的 `/sap/bc/zrest_training` 分類節點，這題新增子節點：

1. SICF 交易碼，在 `zrest_training` node 上右鍵 → **New Sub-Element**，Service Name 填 **rs11**
2. Handler List 掛 **ZCL_RS11_APP**
3. Activate Service
4. 用 `SPROX_HTTP_REQUEST` 測試（**URL 一律填完整網址**）：

第一步，先用 `testrun=true` 預覽（不會真的建立訂單）： `` **POST**

```
https://erpdemo01.itts.com.tw:44300/sap/bc/zrest_training/rs11/salesorders?
testrun=true&sap-client=130 `` `json { "header": { "docType": "TA", "salesOrg": "1710",
"distrChan": "10", "division": "00", "soldTo": "0017100002", "items": [ { "material":
"F-10A", "quantity": 10, "unit": "ST" }, { "material": "F-10A", "quantity": 5, "unit":
"ST" } ] } } ``
```

- 確認回應是 **200**，`testrun:true`，`messages` 陣列裡沒有 **E/A** 等級的訊息文字（如果有，代表欄位對應有問題，先別急著做下一步）

第二步，拿掉 `testrun`，正式建立： `` **POST**

```
https://erpdemo01.itts.com.tw:44300/sap/bc/zrest_training/rs11/salesorders?sap-
client=130 `` ( body 同上 )
```

- 確認回應是 **201**，`salesDocument` 有值，`testrun:false`

- 用 SE16 / ADT Data Preview 查 **VBAK/VBAP**，確認這個 **VBELN** 真的寫進資料庫、**VBAP** 底下有兩筆對應的品項 (項次 000010、000020)

錯誤情境：

- **header** 缺 **soldTo**：確認回 **400** + `{"errorCode":"VALIDATION_FAILED",...}`
- **header.items** 給空陣列 `[]`：確認回 **400** + **VALIDATION_FAILED**
- **material** 給一個不存在的物料號：確認回 **422** + `{"errorCode":"SALES_ORDER_REJECTED","message":"...BAPI 的錯誤訊息..."}`

預期輸出 (範例)

POST /salesorders?testrun=true：HTTP 狀態碼 **200**

```
{
  "salesDocument": "",
  "testrun": true,
  "messages": ["<BAPI 回傳的訊息，例如「將建立標準訂單」之類的提示>"]
}
```

POST /salesorders (正式建立)：HTTP 狀態碼 **201**

```
{
  "salesDocument": "0000012345",
  "testrun": false,
  "messages": ["<BAPI 回傳的訊息>"]
}
```

驗證失敗：HTTP 狀態碼 **400**

```
{"errorCode":"VALIDATION_FAILED","message":"header 的 docType、salesOrg、distrChan、division、soldTo 都必填"}
```

BAPI 拒絕：HTTP 狀態碼 **422**

```
{"errorCode":"SALES_ORDER_REJECTED","message":"<BAPI RETURN 表格裡 E/A 等級訊息串接>"}
```

團隊實務備註

- **XXX_IN/XXX_INX** 成對參數是一整個系列 **BAPI** 的共通設計，不是 **BAPI_SALESORDER_CREATEFROMDAT2** 自己發明的模式——採購單、生產訂單等「Header+Item」類型的標準 BAPI 大多是這個設計，學會這題等於學會一個可以套用到很多其他 BAPI 的通用技能
- **422 Unprocessable Entity** 這個狀態碼在這版 **CL_REST_STATUS_CODE** 沒有對應常數，程式裡直接寫字面值 **422**——**CL_REST_STATUS_CODE** 本質上只是一份「先幫你定義好的 HTTP 狀態碼常數清單」，不是驅動驗證的框架邏輯，沒有常數不代表不能用該狀態碼，缺常數時直接寫字面值即可
- 這題沒有實作對應的 **GET /salesorders/{vbeln}**——刻意聚焦在「巢狀 JSON → 一次 BAPI 呼叫」這個單一教學重點，不做完整 CRUD；如果想要串接查詢功能，可以參考

BAPI_SALESORDER_GETSTATUS/BAPISDH1 或直接查 VBAK/VBAP，但那是另一個題目的範圍

- 把 **TESTRUN** 暴露成 **API** 查詢參數這個手法有其代價：它讓呼叫端可以隨意「預演」但不留痕跡，如果 API 有存取控制或稽核需求，通常需要額外考慮「誰可以用 testrun」「testrun 呼叫要不要記 log」這類問題，這題為了教學單純沒有處理
- **header** 這層 **key** 本身要不要存在，是有爭議的設計選擇，不是唯一解：這題目把 **docType/salesOrg/soldTo** 這些抬頭欄位包在 **header** 物件底下（連 **items** 也巢狀在 **header** 裡），但另一派更「純 REST」的設計主張連 **header** 這層 wrapper 都該拿掉，直接把這些欄位攤平在 JSON 最外層、**items** 變成跟它們同層的一個陣列欄位（`{"docType": "...", "items": [...]}`，不是 `{"header": {"docType": "...", "items": [...]}}`）——攤平版本對前端來說少一層要拆的殼（**order.docType** 而不是 **order.header.docType**），語意上也更像「這些欄位本來就是這個資源的屬性」。這題選擇保留 **header** 分組，主要原因是這樣的結構跟 **BAPISDHD1** 的欄位分組直接對應，設計/除錯時比較容易對照 BAPI 介面文件；如果之後要新增一題刻意示範「更純的攤平式 REST 設計」，可以拿掉 **header** 這層當作對照組
- 這題只示範「一張訂單（1 個 **header**）+ 多筆品項」，沒有涵蓋「一次送多張訂單（多個 **header**，每個各自帶自己的 **items**）」這種批次情境——如果要支援後者，JSON body 的根節點會變成一個陣列，陣列裡每個元素才是「一張訂單」（`[{"docType": "...", "items": [...]}]`，`[{"docType": "...", "items": [...]}]`），這跟 rs10「陣列裡每筆是扁平的單一資源」不一樣，是「陣列裡每筆自己還有巢狀結構」——這是這門課目前完全沒示範過的第三種陣列模式（rs03 是輸出扁平陣列、rs10 是輸入扁平陣列、這題是輸入單一巢狀物件），值得當作下一題延伸的候選主題

思考題

1. 如果 **items** 陣列裡有 5 筆，其中第 3 筆的物料號不存在，**BAPI_SALESORDER_CREATEFROMDAT2** 會是「整張訂單都不建立」還是「跳過那一筆，其他 4 筆照常建立」？這跟 rs10 **BAPI_FLBOOKING_CREATEFROMDATA** 逐筆呼叫、逐筆獨立成敗的行為有什麼本質上的不同？（提示：想想「一次呼叫處理一個完整單據」跟「呼叫端自己迴圈呼叫 N 次」在失敗處理粒度上的差異——這題的 BAPI 呼叫是以整張單據為單位的一次性 LUW）
2. 這題的 **ORDER_HEADER_INX/ORDER_ITEMS_INX** 目前是「有填值的欄位就打 X」，如果這支 API 未來要支援 **PUT**（修改既有銷售訂單），**UPDATEFLAG** 該從 'I' 改成什麼？X 旗標的填法邏輯需要跟著改變嗎？
3. **soldTo** 目前只當作 Sold-to Party（角色 **AG**）填入 **ORDER_PARTNERS**，正式的銷售訂單通常還會有 Ship-to Party（**WE**）、Bill-to Party（**RE**）等其他夥伴角色——如果要讓呼叫端也能指定這些角色，**ty_header** 或 JSON 請求格式該怎麼調整？

答案

見 `zcx_rs11_salesorder_error.clas.abap`、`zcl_rs11_salesorder_service.clas.abap`（+ `.testclasses.abap`）、`zcl_rs11_app.clas.abap`、`zcl_rs11_salesorders.clas.abap`（SAP 端物件 **ZCX_RS11_SALESORDER_ERROR** / **ZCL_RS11_SALESORDER_SERVICE** / **ZCL_RS11_APP** / **ZCL_RS11_SALESORDERS**）。SICF Service 路徑 `/sap/bc/zrest_training/rs11`。